

Edge Geography is XNLP-hard for Pathwidth and in XP for Tree-Partition Width

Thobias Kvalvik Høivik · Erlend Raa Vågset
Western Norway University of Applied Sciences

ESA 2026 · L'Aquila

Edge Geography

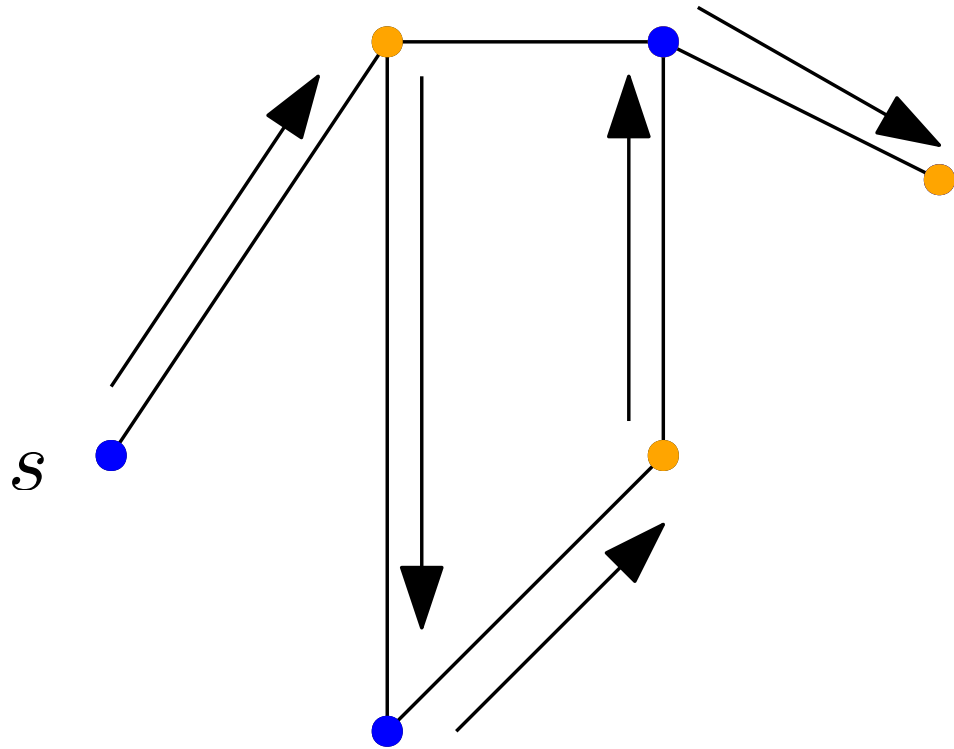
A token starts at a vertex s .

Players alternate:

1. choose an unused edge from the current vertex,
2. move the token along it,
3. delete that edge.

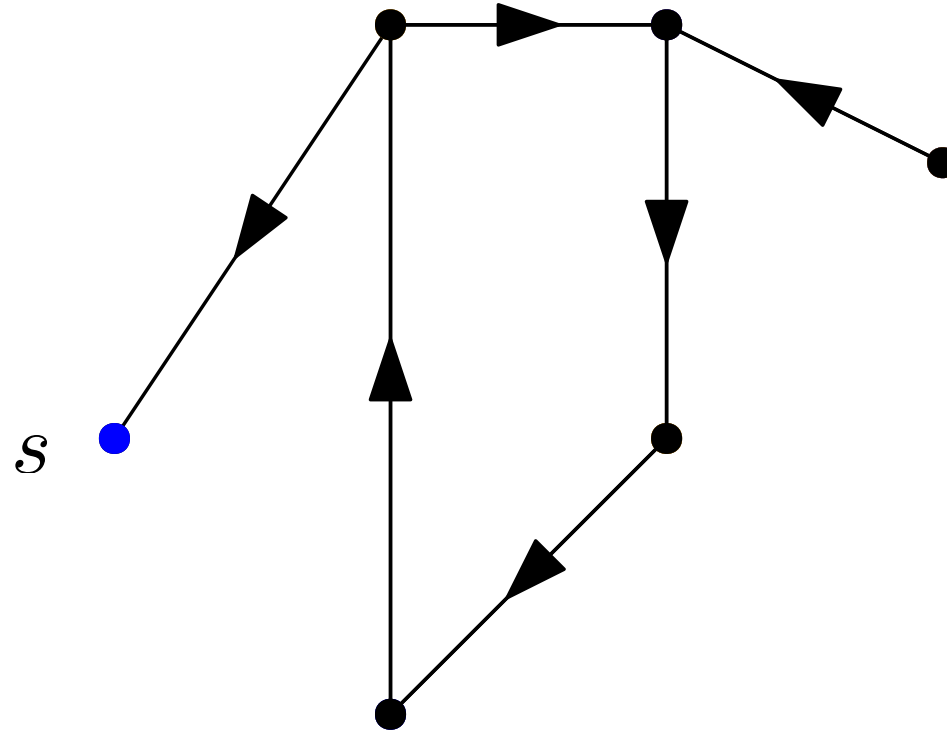
The first player unable to move loses.

Undirected Edge Geography



Player 1 wins!

Directed Edge Geography



Player 2 wins!

Why should width help?

Small separators often suggest small dynamic-programming states.

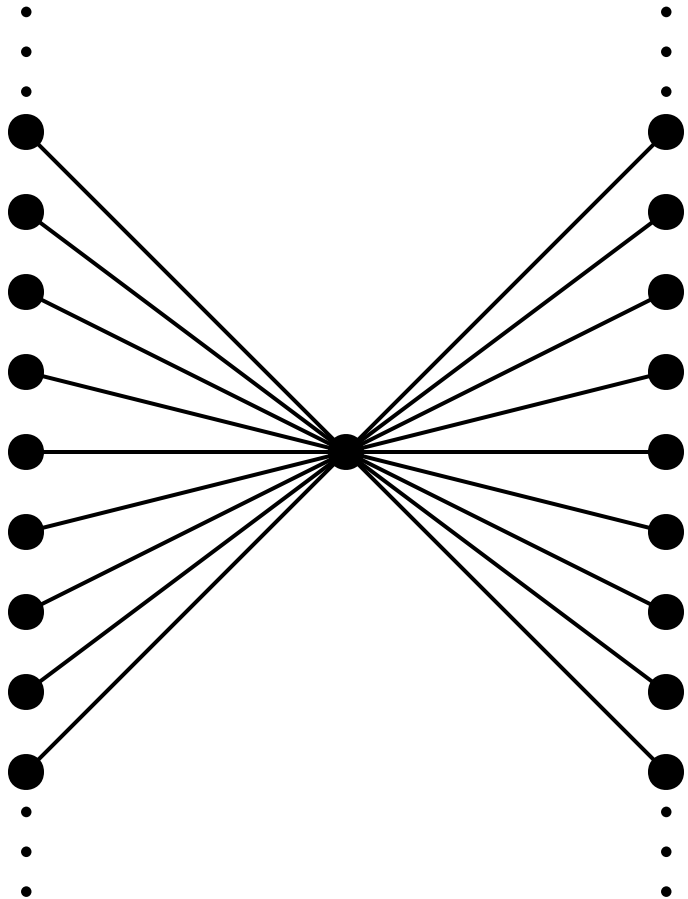
Past | separator | future

But Edge Geography has a catch:

The game deletes **edges**, not vertices.

A separator vertex may be revisited many times while different incident edges disappear.

Arbitrarily many crossings



when $\Delta(G)$ is unbounded!

What width alone can and cannot do

pathwidth

XNLP-hard

treewidth + degree

FPT

tree-partition width

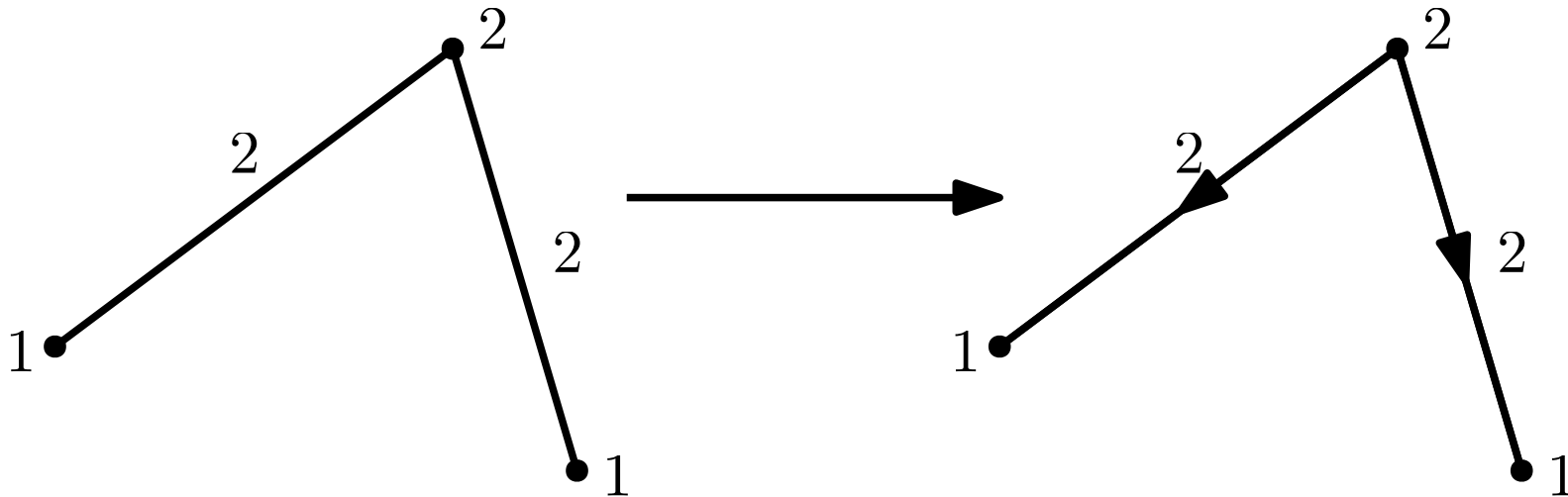
XP

Main question: **which decompositions support a compact summary of residual play?**

Source problem: Chosen Maximum Outdegree

Choose an orientation of every edge so that each vertex respects its budget.

$$\sum_{e \in \theta_w(v)} w(e) \leq t(v) \quad \forall v$$



Known XNLP-complete when parameterized by pathwidth.

Reduction idea in two phases

Phase I — Player 1 encodes an orientation

For each input edge $e = \{x, y\}$, Player 1 chooses one of two sides.

$$\text{choose the } y\text{-side} \iff x \rightarrow y$$

Phase II — Player 2 challenges a vertex

Player 2 chooses x and tests the total weight oriented out of x .

If outgoing weight $> t(x)$, Player 2 has one excursion too many.

How a gadget remembers an orientation

put wicked sick picture here

During the initial traversal, one side is consumed.

Later, in the challenge phase:

used side

challenging it is losing

unused side

one successful challenge excursion

TODO visual: simple side-selection gadget with “used” side greyed out and “unused” side highlighted.

Why correctness follows

Feasible orientation

$$\sum_{e \in \theta_\omega(x)} w(e) \leq t(x) \text{ for every } x.$$

Player 2 eventually runs out of non-losing challenges.

Player 1 wins

Infeasible orientation

For some x ,

$$\sum_{e \in \theta_\omega(x)} w(e) > t(x).$$

Player 2 chooses this x and survives one challenge too many.

Player 2 wins

Why pathwidth is preserved

Process an input edge $e = \{x, y\}$ when x and y coexist in the source path decomposition.

Keep only the necessary anchors alive.

Introduce auxiliary gadget vertices locally, then forget them immediately.

$$\text{pw}(G') = O(k)$$

TODO visual: path-decomposition timeline. Show x, y intervals, a green local gadget block, short-lived auxiliary vertices.

Directed to undirected

Each directed edge is replaced by a constant-size pseudoarc.

$$u \rightarrow v \quad \rightsquigarrow \quad \text{undirected pseudoarc}$$

Safe traversal works from u to v .

Entering the wrong way is losing under optimal play.

Directed hardness transfers to Undirected Edge Geography.

Tree partitions give controlled interfaces

A rooted tree partition groups vertices into bags.

Edges are either:

- inside one bag, or
- between adjacent bags of the tree.

For width k , a parent-child cut has at most k^2 ports.

A child subtree can only interact with its parent through boundedly many ports.

TODO visual: parent bag with several child bags; highlight ports crossing one parent-child boundary.

Compress child subtrees by type

Think of each child subtree as a black box.

An interface type records:

- where play can enter,
- where it may return,
- what residual child remains,
- whose turn it is after returning.

For fixed k , there are only finitely many types.

$$A, A, A, B, C \rightsquigarrow 3A + B + C$$

TODO visual: several child subtrees collapsed to labelled boxes, then compressed into multiplicities.

Bottom-up dynamic programming

At each bag, a compressed residual configuration is roughly:

$$\Gamma = (U, F, m)$$

unused
ports

unused
internal edges

multiplicities
of child types

Local games are acyclic because every move consumes an edge or reduces a residual child type.

This gives an $n^{f(k)}$ -time algorithm: XP.

Takeaway

Pathwidth

small separators

**but residual history is
uncontrolled**

XNLP-hard

Tree-partition width

bounded interfaces

residual play can be compressed

XP

Open: Is Edge Geography in XP parameterized by pathwidth alone?

Backup: result table

Parameter	Directed	Undirected
pathwidth	XNLP-hard	XNLP-hard
treewidth + max degree	FPT	follows via known results / transfer context
tree-partition width	XP	XP

TODO: verify exact phrasing for the undirected/treewidth+degree cell before final deck. This is a backup slide, not part of the core talk.

Backup: edge gadget behavior

During initial traversal:

- Player 1 chooses a side.
- Player 2's replies are forced.
- play exits at the return vertex.

During challenge:

- challenging the used side is losing,
- challenging the unused side consumes one challenge edge and one return path.

TODO visual: cleaned-up Figure 3 or a custom simplified gadget diagram.

Backup: pathwidth bookkeeping

The constructed decomposition follows the source path decomposition.

For a bag B_i :

- keep challenge anchors for vertices currently in B_i ,
- realize edge gadgets in local contiguous blocks,
- introduce auxiliary vertices only briefly,
- realize return paths just before forgetting their vertex.

Only $O(k)$ long-lived vertices are needed.